

Version Control: Git and GitHub

Git is a distributed version control system that helps track changes in code, enabling collaboration and rollback to previous versions.

GitHub is a cloud-based platform that hosts Git repositories and allows developers to collaborate, review code, and manage projects.

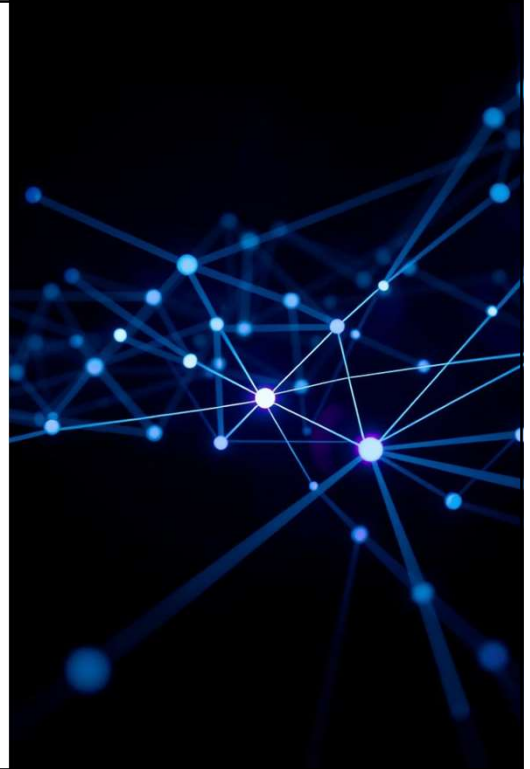
Together, they make version control easier, especially for teams working on software development.

Presented by:

D. Haripriya

22R21A66E4

III CSM-C



What is Version Control?



Track Changes

Systems track changes to files over time. This creates a complete history.



Revert & Compare

You can easily revert to previous versions. Compare changes between any two points.



Collaborate Safely

It enables seamless team collaboration. Version control prevents conflicts effectively.



Why Use Version Control?



Detailed History

Maintain a comprehensive record of all modifications. See who changed what and when.



Team Collaboration

Multiple developers can work concurrently. Integrate changes without overwriting.



Backup & Recovery

Safeguard against accidental data loss. Easily restore any past version.



Experimentation

Create isolated branches for new features. Develop without affecting the main project.

Introduction to Git

Distributed System

Git is a powerful distributed distributed version control system. Each developer has a full copy.

Linus Torvalds

It was developed by Linus Torvalds in 2005. This was to manage Linux kernel development.

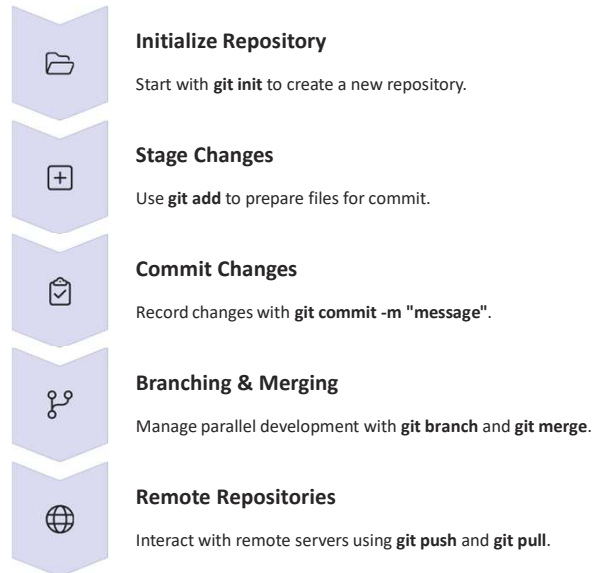
Widespread Adoption

Git is widely used in open-source and enterprise projects. It is a industry standard.

Core Concepts

Understand repositories, commits, branches, and merges. These are fundamental to Git.

Git Workflow



Introduction to GitHub



Web Platform

GitHub is a web-based platform. It enhances version control and collaboration.

Host Repositories

It hosts Git repositories. Provides a centralized location for projects.

Collaboration Tools

Offers features like pull requests and issue tracking. Facilitates team interaction.

Collaborating with GitHub

Forking

Create a personal copy of a repository. Work independently without affecting the original.

Issue Tracking

Manage bugs and feature requests efficiently. Organize tasks and prioritize work.



Pull Requests

Propose your changes to the main repository. This initiates a discussion process.

Code Review

Team members discuss and approve changes. Ensures code quality and consistency.

Best Practices for Git and GitHub



Clear Commits

Write concise, descriptive commit messages. Explain **why** changes were made.



Use Branching

Always use branches for new features and bug fixes. Keep main branch stable.



Pull Regularly

Keep your local repository updated. Pull changes from remote often.



Code Review

Always review code before merging pull requests. Catch errors early.

Git vs. GitHub

Git: The Tool

Git is the **software** itself. It's a command-line tool.

- Distributed version control system.
- Local repository on your machine.
- Manages code changes offline.
- Core functionality: commit, merge, branch.

GitHub: The Platform

GitHub is a **service** that uses Git. It's web-based.

- Hosts Git repositories online.
- Adds collaboration features.
- Provides GUI for Git operations.
- Offers issue tracking, pull requests.

THANK YOU!